Lab Assessment Rubrics of Digital System Design (CPE-344)

Lab Number	**11**
Lab Title	Implementation of 4 bit shift register and majority counter
Submitted by	
Names	**Registration Number**
Sumaira Safeer	FA22-BCE-019

Rubrics name & number		**Marks**	
		In-Lab	**Post-Lab**
Engineering Knowledge	***R2: Use of Engineering Knowledge and follow Experiment Procedures:*** *Ability to follow experimental procedures, control variables, and record proceduralsteps on lab report.*		
	R3: Interpretation of Subject Knowledge: *Ability to interpret and explain mathematical and/or visual forms, includingequations, diagrams, graphics, figures, and tables.*		
Problem Analysis	***R6: Experimental Data Analysis:*** *Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.*		
Design	***R7: Implementing Design Strategy:*** *Ability to execute a solution taking into consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram]*		
	R8: Best Coding Standards: *Ability to follow the coding standards and programming practices.*		
Modern Tools Usage	***R9: Understand Tools:*** *Ability to describe and explain the principles behind and applicability ofengineering tools.*		
	R11: Tools Evaluation: ***Ability to simulate the experiment and them using hardware tools to verify*** ***The results.***		
Individual and Teamwork	***R12: Individual Work Contributions:*** *Ability to carry out individual responsibilities.*		
	R13: Management of Teamwork: *Ability to appreciate, understand and work with multidisciplinary teammembers.*		

Rubrics to follow

Rubrics	***R2***	***R3***	***R5***	***R6***	***R7***	***R8***	***R9***	***R11***	**R12**	**R13**
In-Lab										
Post-Lab										

Lab: 11

1. Objectives:

The objective of this lab is to get familiar with

- Implementation of Task in Verilog
- Implementation of Function in Verilog

2. Lab Tasks:

2.1. Task#01:

Alu Designing using Task

Solution:

- **Verilog Code**

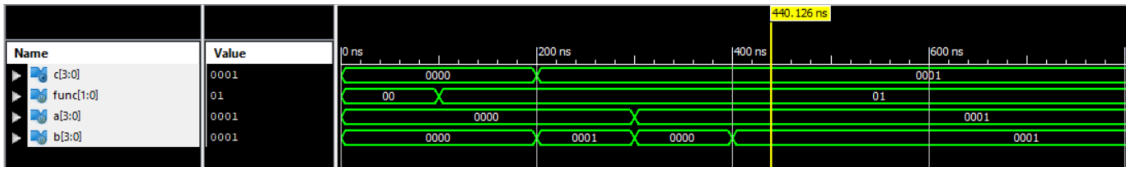
```
module aluUsingTask(func, a, b, c);

input [1:0]func;
input [3:0]a, b;
output [3:0]c;
reg[3:0]c;

task my_and;
input [3:0]a, b;
output [3:0]andout;
integer i;

begin
for (i =3; i>=0; i= i-1)
andout[i]=a[i]&b[i];
end
endtask

always@(func or a or b)
begin
case(func)
2'b00:my_and(a, b, c);
2'b01:c=a|b;
2'b10: c=a-b;
default: c=a+b;
endcase
end

endmodule
```

- **Verilog Test Bench**

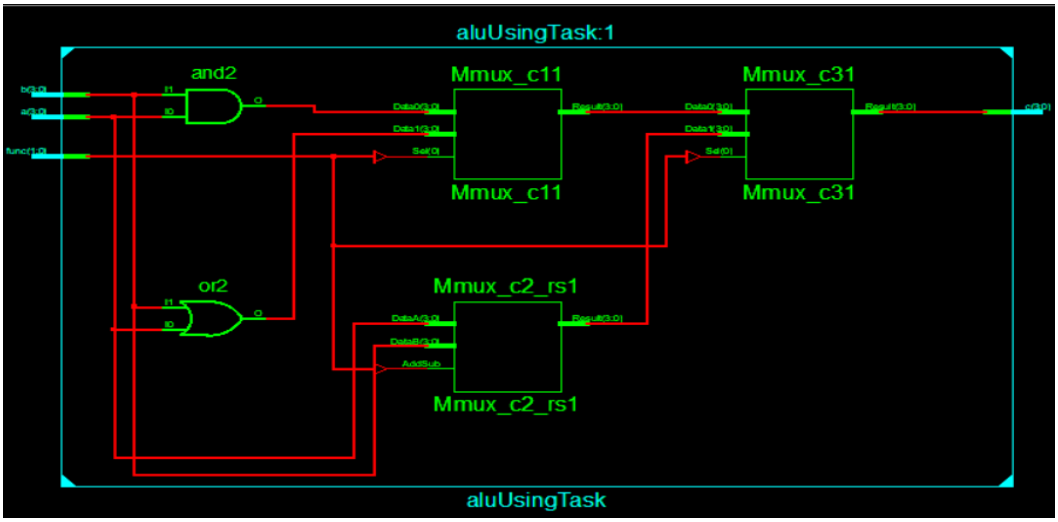
```
module aluTestBench;

    // Inputs
    reg [1:0] func;
    reg [3:0] a;
    reg [3:0] b;

    // Outputs
    wire [3:0] c;

    // Instantiate the Unit Under Test (UUT)
    aluUsingTask uut (
        .func(func),
        .a(a),
        .b(b),
        .c(c)
    );

    initial begin
        // Initialize Inputs
        func = 0;
        a = 0;
        b = 0;

        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        // Initialize Inputs
        func = 1;
        a = 0;
        b = 0;

        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        // Initialize Inputs
        func = 1;
        a = 0;
        b = 1;
```

```
        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        func = 1;
        a = 1;
        b = 0;

        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        func = 1;
        a = 1;
        b = 1;

        // Wait 100 ns for global reset to finish
        #100;
    end

endmodule
```

- **Waveform**



- **RTL Diagram**



2.2. Task#02:

Designing RCA using task

Solution:

- **Verilog Code**

```
 module rcaUsingTask(input wire[3:0] a, input wire[3:0] b, input wire c_in, output
reg c_out, output reg [3:0] sum);
reg carry[4:0];
integer i;

task FA(input in1, input in2, input carry_in, output reg out, output carry_out);
{carry_out, out} = in1 + in2 + carry_in;
endtask

always @(*)
begin
carry[0] = c_in;
for(i = 0; i<4; i= i+1)
begin
FA (a[i], b[i], carry [i], sum[i], carry[i+1]);
end
c_out = carry [4];
end

endmodule
```

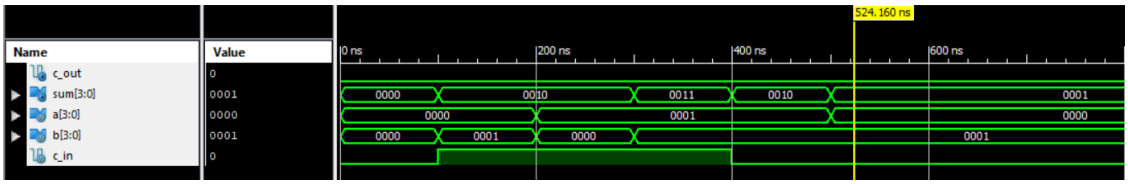
- **Verilog Test Bench**

```
module rcaTestBenchUsingTask;

    // Inputs
    reg [3:0] a;
    reg [3:0] b;
    reg c_in;

    // Outputs
    wire c_out;
    wire [3:0] sum;
```

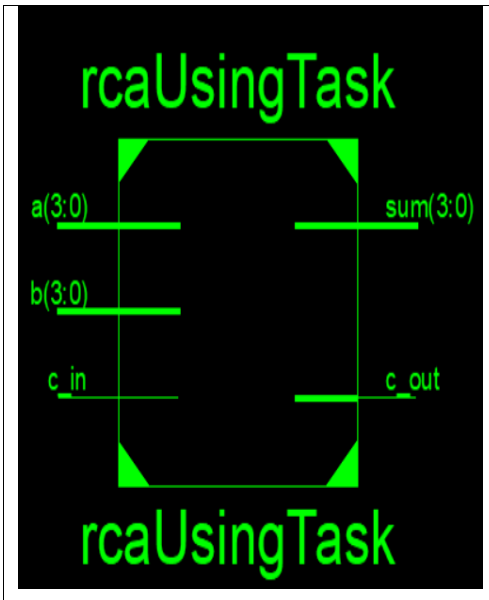
```
// Instantiate the Unit Under Test (UUT)
rcaUsingTask uut (
        .a(a),
        .b(b),
        .c_in(c_in),
        .c_out(c_out),
        .sum(sum)
);

initial begin
        // Initialize Inputs
        a = 0;
        b = 0;
        c_in = 0;

        // Wait 100 ns for global reset to finish
        #100;

        // Add stimulus here
        // Initialize Inputs
        a = 0;
        b = 1;
        c_in = 1;

        // Wait 100 ns for global reset to finish
        #100;
        // Initialize Inputs
        a = 1;
        b = 0;
        c_in = 1;

        // Wait 100 ns for global reset to finish
        #100;
        // Initialize Inputs
        a = 1;
        b = 1;
        c_in = 1;

        // Wait 100 ns for global reset to finish
        #100;
        // Initialize Inputs
        a = 1;
```

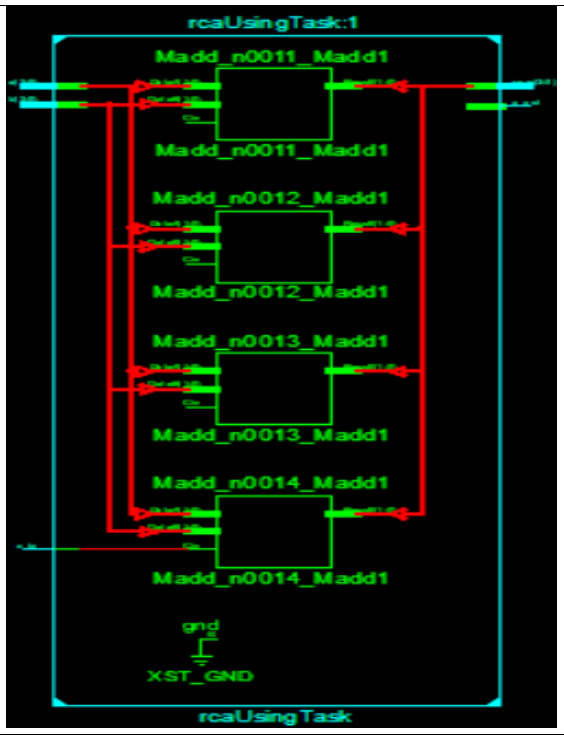
```
                b = 1;
                c_in = 0;

                // Wait 100 ns for global reset to finish
                #100;
                // Initialize Inputs
                a = 0;
                b = 1;
                c_in = 0;


            end
        endmodule
```

- **Waveform**



- **RTL Diagram**





2.3. Task#03

Design 4x1 using 2x1 function

Solution:

- **Verilog Code**

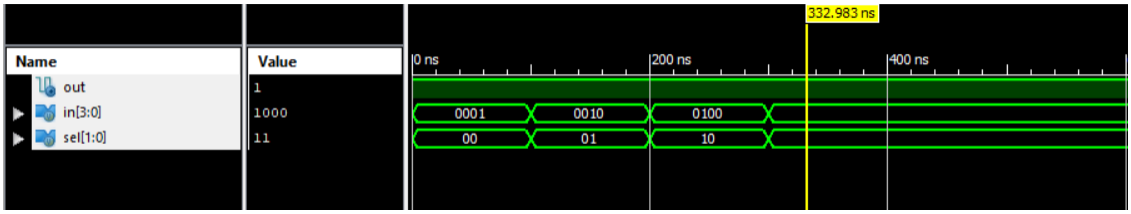
```
 module MUX4to1(input [3:0] in, input [1:0] sel, output out);
wire out1, out2;

function MUX2to1;
input in1, in2;
input select;
MUX2to1 = select ? in2:in1;
endfunction

assign out1 = MUX2to1(in[0], in[1], sel[0]);
assign out2= MUX2to1(in[2], in[3], sel[0]);
assign out= MUX2to1(out1, out2, sel[1]);

endmodule
```

- **Verilog Test Bench**

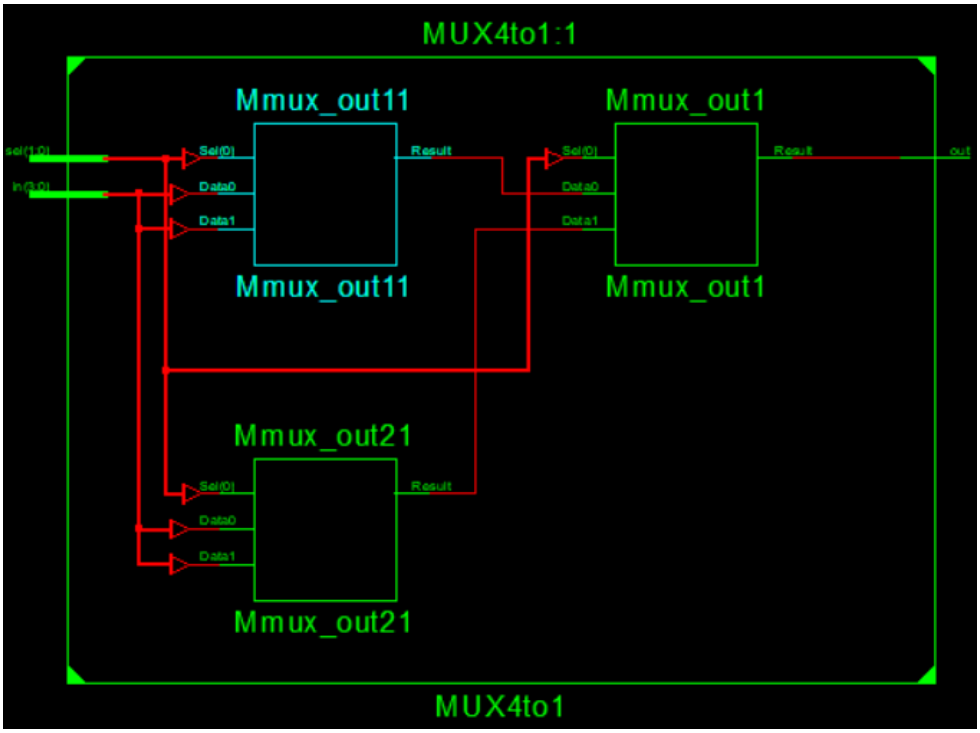
```
module muxUsingFunction;
        // Inputs
        reg [3:0] in;
        reg [1:0] sel;

        // Outputs
        wire out;

        // Instantiate the Unit Under Test (UUT)
        MUX4to1 uut (
                .in(in),
                .sel(sel),
                .out(out)
        );

        initial begin
                // Initialize Inputs
                in = 4'b0001;
                sel = 2'b00;

                // Wait 100 ns for global reset to finish
                #100;

                // Add stimulus here
```

```
                    in = 4'b0010;
                    sel = 2'b01;

                    // Wait 100 ns for global reset to finish
                    #100;
                    // Initialize Inputs
                    in = 4'b0100;
                    sel = 2'b10;

                    // Wait 100 ns for global reset to finish
                    #100;
                    // Initialize Inputs
                    in = 4'b1000;
                    sel = 2'b11;

                    // Wait 100 ns for global reset to finish
                    #100;
                end
endmodule
```

- **Waveform**



- **RTL Diagram**



2.4. Post Lab Task#01

64 Bit Comparator Using 8 Bit with Task

Solution:

- **Verilog Code**

```
 module Compareator();
input [63:0]Num1,Num2;
output Greater,Equaler,Lesser;
wire [7:0]G,E,L;
wire [5:0]Great,Equal,Less;
//////////////TASK/////////
task Compare;
input wire [7:0]A,B;
output reg G,E,L;
begin
if(A>B)
begin
G = 1'b1; L = 1'b0; E = 1'b0;
end
else if(A<B)
begin
G = 1'b0;L = 1'b1;E = 1'b0;
end
else
 begin
```

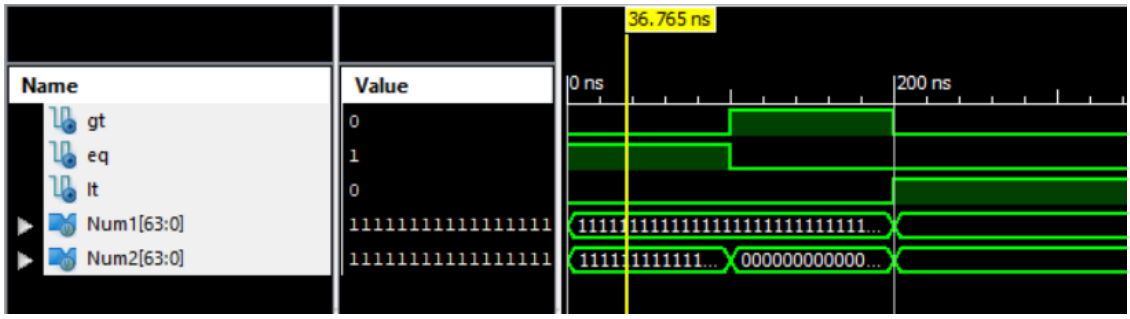
```
G = 1'b0; L = 1'b0;E = 1'b1;
end
end
endtask
Compare(Num1[7:0],Num2[7:0],G[0],E[0],L[0]);
Compare(Num1[15:8],Num2[15:8],G[1],E[1],L[1]);
Compare(Num1[23:16],Num2[23:16],G[2],E[2],L[2]);
Compare(Num1[31:24],Num2[31:24],G[3],E[3],L[3]);
Compare(Num1[39:32],Num2[39:32],G[4],E[4],L[4]);
Compare(Num1[47:40],Num2[47:40],G[5],E[5],L[5]);
Compare(Num1[55:48],Num2[55:48],G[6],E[6],L[6]);
Compare(Num1[63:56],Num2[63:56],G[7],E[7],L[7]);
// Creation Of 16 Bit Result
assign Equal[0]=E[0]&E[1];
assign Equal[1]=E[2]&E[3];
assign Equal[2]=E[4]&E[5];
assign Equal[3]=E[6]&E[7];
assign Great[0]=(E[1]&G[0])|G[1];
assign Great[1]=(E[3]&G[2])|G[3];
assign Great[2]=(E[5]&G[4])|G[5];
assign Great[3]=(E[7]&G[6])|G[7];
assign Less[0]=(E[1]&L[0])|L[1];
assign Less[1]=(E[3]&L[2])|L[3];
assign Less[2]=(E[5]&L[4])|L[5];
assign Less[3]=(E[7]&L[6])|L[7];
//Creation Of 32 Bit Results
assign Equal[4]=Equal[0]&Equal[1];
assign Equal[5]=Equal[2]&Equal[3];
assign Great[4]=(Equal[1]&Great[0])|Great[1];
assign Great[5]=(Equal[3]&Great[2])|Great[3];
assign Less[4]=(Equal[1]&Less[0])|Less[1];
assign Less[5]=(Equal[3]&Less[2])|Less[3];
//Creation Of 64 Bit Results
assign Equaler = Equal[4]&Equal[5];
assign Greater= (Equal[5]&Great[4])|Great[5];
assign Lesser= (Equal[5]&Less[4])|Less[5];
endmodule
```

- **Verilog Test Bench**

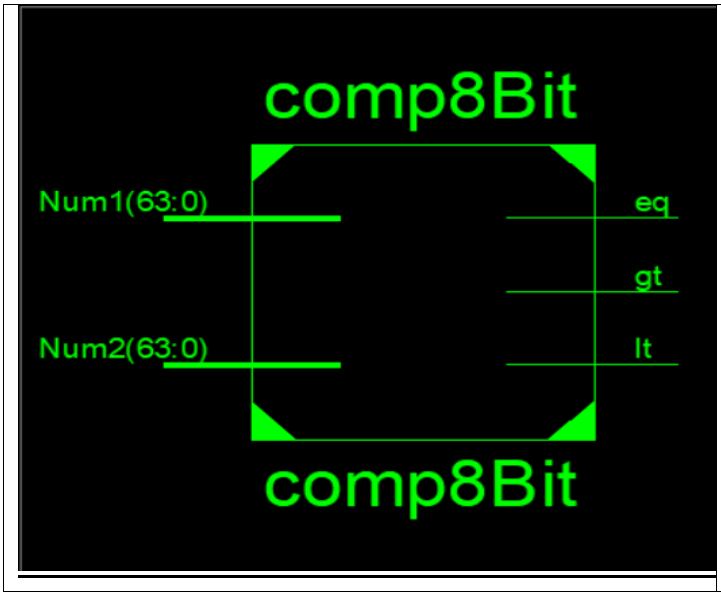
```
module testBench;
        reg [63:0] Num1;
        reg [63:0] Num2;
```

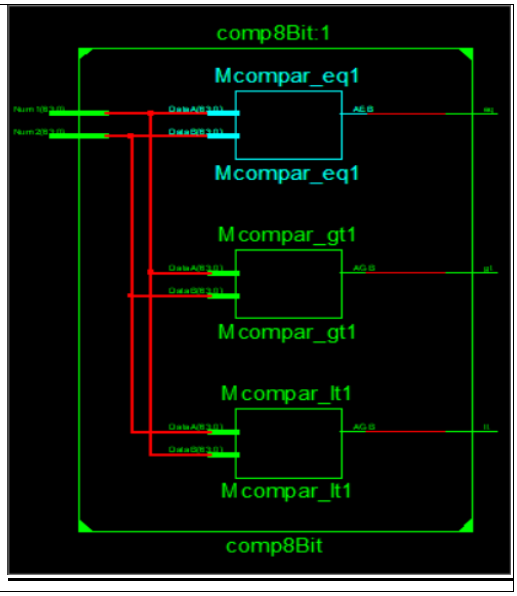
```
        wire gt;
        wire eq;
        wire lt;
        comp8Bit uut (
                .Num1(Num1),
                .Num2(Num2),
                .gt(gt),
                .eq(eq),
                .lt(lt)
        );
        initial begin
                Num1 = 64'b
1111111111111111111111111111111111111111111111111111111111111111;
                Num2 = 64'b
1111111111111111111111111111111111111111111111111111111111111111;
                #100;
                Num1 = 64'b
1111111111111111111111111111111111111111111111111111111111111111;
                Num2 = 64'b
0000000000000000000000000000000000000000000000000000000000000000;
                #100;
                Num1 = 64'b
0000000000000000000000000000000000000000000000000000000000000000;
                Num2 = 64'b
1111111111111111111111111111111111111111111111111111111111111111;
                #100;
        end
endmodule
```

- **Waveform**



- **RTL Diagram**


comp8Bit
Num1(63:0)
Num2(63:0)
eq
gt
lt
comp8Bit


comp8Bit:1
Mcompar_eq1
Mcompar_eq1
Mcompar_gt1
Mcompar_gt1
Mcompar_lt1
Mcompar_lt1
comp8Bit